

CORE 06 — Database Management Systems (DBMS)

Shared-core file 6 of the NPSC CTSE 2026 prep set.

Three depths for every topic: **one-line fact** (MCQ), **5-mark answer** (~5–6 points),

10/15-mark answer (worked explanation + diagram).

0. Why this matters — where DBMS appears in your three papers

Elective	Where	How heavy
CS Degree	P-II §3 — "Data Abstraction, Data models, Data independence, DDL, Attributes, Keys, Query processing, Structure of relational Databases, Example of SQL; Distributed Databases, Security and integrity, Network Model." Names data independence, keys, the network model explicitly.	Heavy
CS Diploma	P-II §3 — Data vs Information, Database, DBMS, DBA, functions of DBMS : relational system/model/optimization, base tables and views; relational data objects (domains, kinds of relations, relations and predicates); the E/R model and E/R diagrams ; SQL (data definition, manipulation, retrieval, update, table expressions, conditional/scalar expressions, embedded SQL); database security (authentication, authorization, access control, enforcement).	Heavy
Computer Forensic	TP-II §5 — "E-R diagrams and their transformation to relational design, normalization. SQL: Data Definition and Data Types; Constraints, Queries, Insert/Delete/Update; Views, Stored Procedures and Functions; Database Triggers, SQL Injection , Transaction Processing, Concurrency Control, Database Recovery, Object and Object-relational Database; Database Security and Authorization, data dictionary." The only paper that names SQL injection , triggers , recovery and concurrency by name.	Very heavy

This is a top-three shared topic. All three electives name E-R diagrams, relational design, keys, SQL and security; two name normalization; the Forensic paper adds transactions, concurrency, recovery and SQL injection. Study it once, answer it three times. The numerical parts — normalization decompositions, candidate-key finding, relational algebra, precedence graphs — are the highest-return practice you can do, because worked answers are unambiguous to mark.

Topic checklist

- [] Data vs information; the database and the DBMS
- [] Functions of a DBMS; role of the DBA
- [] DBMS vs the traditional file system — every advantage
- [] **Three-schema (ANSI/SPARC) architecture** and the two data independences
- [] Data models: hierarchical, network, relational, ER, object, object-relational
- [] **E-R model**: entities, attribute types, relationships, degree, cardinality, participation
- [] Weak entities; generalisation, specialisation, aggregation
- [] **E-R → relational schema** — all seven mapping rules with a worked example
- [] Relational model terms: relation, tuple, attribute, domain, degree, cardinality
- [] **Keys**: super, candidate, primary, alternate, foreign, composite, surrogate
- [] Integrity constraints: domain, entity, referential, key
- [] **Relational algebra** — every operator with a worked query
- [] FDs, **Armstrong's axioms**, **attribute closure**, **finding candidate keys**
- [] **1NF, 2NF, 3NF, BCNF, 4NF** — each with a worked decomposition
- [] Lossless-join and dependency-preservation tests

- [] **SQL**: DDL, DML, DCL, TCL
- [] **All join types** with example queries and output
- [] Subqueries, aggregates, GROUP BY / HAVING, views, indexes
- [] Stored procedures and triggers
- [] **Transactions and ACID**
- [] Schedules, **serializability**, **precedence (conflict) graphs**
- [] Concurrency control: locking, **2PL**, timestamp ordering, deadlock
- [] **Recovery**: log-based (deferred/immediate), checkpoints, ARIES idea
- [] Database security, authorization, GRANT/REVOKE; **SQL injection**
- [] Distributed databases; the **network** and hierarchical models

1. Data, information, the database and the DBMS

Concept

Start with the distinction the Diploma paper opens with. **Data** are raw, unprocessed facts with no context — 36, Kohima, 2026-09-06. **Information** is data that has been **processed, organised and given context** so that it means something — "Kohima recorded 36 mm of rain on 6 September 2026." Information is data made useful for a decision. The one-line test: data is the input, information is the output; **information = data + meaning/processing**.

A step up is **knowledge** (patterns derived from information — "September is Kohima's wettest month") and **wisdom** (judgement applied to knowledge). This is the **DIKW pyramid** (Data → Information → Knowledge → Wisdom).

Term	Meaning	Example
Data	Raw facts, no context	9876543210, Naga, 450
Information	Processed, contextual, meaningful data	"Customer Naga's phone is 9876543210; order total ₹450"
Metadata	Data about data — describes structure and meaning	"Column <code>phone</code> is a 10-digit string; <code>total</code> is currency"
Database	An organised, shared collection of related data	The whole customer/order/product store
DBMS	The software that stores, manages and controls access to the database	Oracle, MySQL, PostgreSQL, SQL Server, MongoDB
Database system	Database + DBMS + applications + users	The complete running system

A **database** is a shared, integrated, self-describing collection of related data plus a description of that data (**metadata**, held in the **data dictionary** / **system catalog**). A **DBMS** is the layer of software between the physical stored data and the users/applications; it lets you **define, construct, manipulate and share** a database while enforcing security and integrity.

Functions of a DBMS

A guaranteed 5-marker in the Diploma paper. Memorise six to eight:

Function	What it does
Data definition	DDL to define schemas, types, structures, constraints — stored as metadata in the catalog

Data manipulation	DML to insert, update, delete and, above all, query data
Data storage & retrieval	Manages physical storage, files, indexes and buffers transparently
Concurrency control	Lets many users access data simultaneously without corrupting it
Transaction management	Guarantees ACID — all-or-nothing, consistent, isolated, durable
Recovery	Restores a consistent state after a crash or failure (logs, backups)
Security & authorization	Controls who may see/change what (users, roles, privileges)
Integrity	Enforces rules (constraints) so data stays valid and consistent
Data dictionary management	Maintains metadata describing the whole database
Multi-user support / data sharing	One integrated store serving many applications and users

The role of the Database Administrator (DBA)

The **DBA** is the person (or team) with central control of the database. Duties:

- **Schema definition and modification** — designing and evolving the logical structure.
- **Storage structure and access-method definition** — physical design, indexes, tuning.
- **Granting and revoking authorization** — deciding who may do what (security policy).
- **Backup and recovery** planning and execution.
- **Performance monitoring and tuning** — query optimisation, index management.
- **Integrity enforcement** — defining and maintaining constraints.
- Liaising with users, capacity planning, managing the DBMS software itself.

Distinguish the DBA (administers the *system*) from the **database designer** (designs the schema) and the **application programmer** (writes programs that use it) and the **end user** (naive/casual/sophisticated).

DBMS vs the traditional file-processing system

Before DBMSs, each application kept its own files. This "file system" approach has classic defects; listing them *as the advantages of a DBMS* is the standard 5/10-mark answer.

Problem with file systems	How a DBMS solves it
Data redundancy — same data (e.g. a customer's address) copied in many files	Controlled redundancy — data integrated and normalised, stored once
Data inconsistency — copies disagree after an update	Consistency — one copy, or synchronised copies via constraints
Difficult data access — every new query needs a new program	Ad-hoc querying via a high-level query language (SQL)
Data isolation — data scattered across incompatible file formats	Integration — one logical, uniform store
Integrity problems — rules buried in program code, hard to change	Declarative constraints enforced centrally by the DBMS
Atomicity problems — a crash mid-update leaves half-done work	Transactions guarantee all-or-nothing (atomicity)

Concurrent-access anomalies — simultaneous updates corrupt data	Concurrency control (locking, timestamps)
Security problems — hard to give per-user, per-field access	Fine-grained authorization (users, roles, views, GRANT/REVOKE)
Program–data dependence — file structure hard-wired into programs	Data independence — structure can change without rewriting programs

The two headline advantages to name first: controlled redundancy and data independence. Add: data sharing, integrity, security, backup/recovery, and enforcement of standards.

Disadvantages of a DBMS (for balance, sometimes asked): higher **cost** (software, hardware, trained staff); **complexity**; larger size/overhead; and a DBMS is **overkill** for tiny, single-user, simple applications where a flat file suffices.

Likely exam questions

- **[5]** Differentiate between data and information with examples.
- **[5]** What is a DBMS? State any six functions of a DBMS.
- **[5]** State the duties/responsibilities of a Database Administrator.
- **[5]** List the disadvantages of a file-processing system that a DBMS overcomes.
- **[10]** Explain the advantages of a DBMS over a traditional file-processing system.
- **[10]** What is a DBMS? Describe its major functions and the roles of the different users of a database system.

MCQ traps

Trap	Correct answer
"Processed data with context is?"	Information. Raw facts = data .
"Data about data is?"	Metadata.
"Metadata is stored in the?"	Data dictionary / system catalog.
"Who grants privileges?"	The DBA .
"Main disadvantage of file systems?"	Data redundancy and inconsistency.
"Which guarantees all-or-nothing?"	Transaction (atomicity), a DBMS feature.
"DIKW pyramid order?"	Data → Information → Knowledge → Wisdom.

2. The three-schema architecture and data independence

Concept

The **ANSI/SPARC three-schema (three-level) architecture** (1975) is the foundation of data independence. Its whole purpose is to **separate the user's view of the data from the way the data is physically stored**, so that a change at one level does not ripple up to the others.

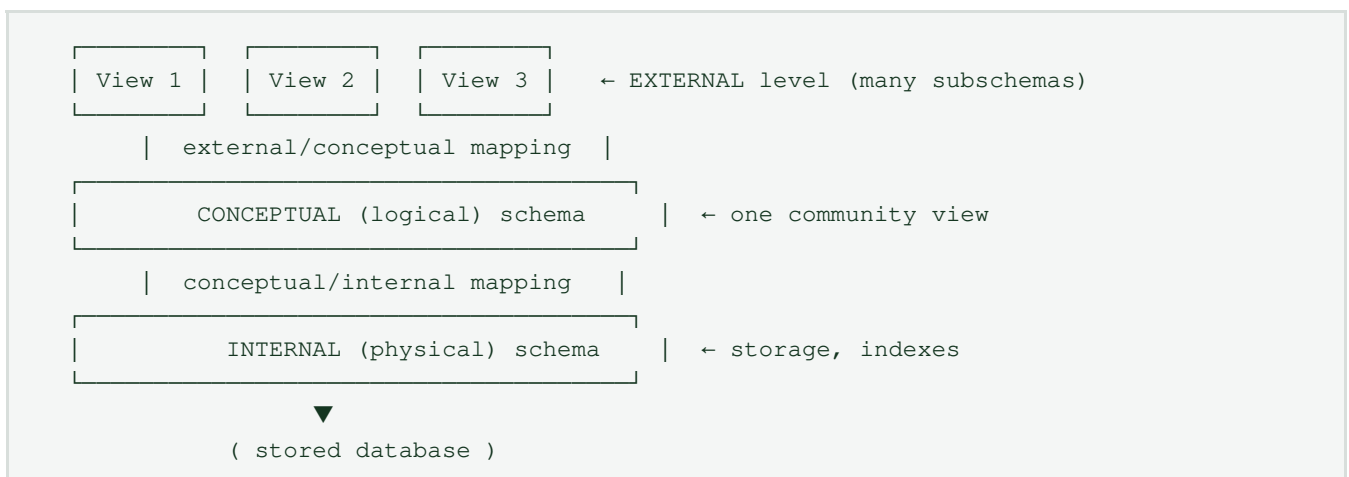
Three levels, from most abstract to most concrete:

Level	Also called	Describes	Concerned with	Who cares

External (top)	View level / subschema	<i>What each user/group sees</i> — many different customised views	Hiding data; presenting a tailored subset	End users, application programmers
Conceptual (middle)	Logical level / community view	<i>What data is stored and the relationships</i> — the whole database logically, once	Entities, attributes, relationships, constraints — no physical detail	DBA, database designer
Internal (bottom)	Physical level	<i>How data is physically stored</i> — files, records, indexes, compression, placement	Storage structures, access paths, performance	System programmers, DBMS

Diagram to hand-draw

Draw three horizontal boxes stacked vertically. **External** at the top, split into several small view boxes (View 1, View 2, View 3). **Conceptual** in the middle as one box. **Internal** at the bottom as one box, sitting on a "stored database" cylinder. Label the join between External and Conceptual "**external/conceptual mapping**" and the join between Conceptual and Internal "**conceptual/internal mapping**".



The two **mappings** between adjacent levels are the machinery that makes independence possible: they let the DBMS translate a request expressed at one level into the level below.

Instance vs schema: the **schema** is the *design/structure* (rarely changes); an **instance** is the *actual data at a moment* (changes constantly). Analogy: schema is a variable's declaration, an instance is its current value.

Data independence — the payoff

Data independence is the capacity to change the schema at one level **without having to change the schema at the next higher level** (and hence without rewriting application programs).

Two kinds, and getting the two straight is the whole question:

	Logical data independence	Physical data independence
Definition	Ability to change the conceptual schema without changing external schemas / applications	Ability to change the internal (physical) schema without changing the conceptual schema
Example change	Add a new attribute or entity, split a table	Add an index, change the file organisation, move to a new disk, change compression
Achieved by	The external/conceptual mapping	The conceptual/internal mapping
Which is harder?	Logical is harder to achieve — applications depend heavily on logical structure	Physical is easier — physical details are already hidden

The exam sentence: *logical data independence is harder to achieve than physical, because user applications are strongly tied to the logical structure of the data.*

Likely exam questions

- **[5]** Explain the three-schema architecture of a DBMS with a diagram.
- **[5]** Differentiate between logical and physical data independence.
- **[5]** Differentiate between a schema and an instance.
- **[10]** Explain the ANSI/SPARC three-level architecture. How does it achieve data independence? Distinguish the two types with examples.

MCQ traps

Trap	Correct answer
"How many levels in ANSI/SPARC?"	Three — external, conceptual, internal.
"View level is the?"	External level.
"Physical storage is described at the?"	Internal level.
"Adding an index without touching the logical schema is?"	Physical data independence.
"Which independence is harder to achieve?"	Logical .
"Schema vs instance — which changes often?"	The instance (data).
"Body of metadata that describes the database?"	Conceptual schema / catalog .

3. Data models

Concept

A **data model** is a set of concepts for describing the structure of a database — the data, the relationships, the constraints and (sometimes) the operations. It is the notation in which a schema is expressed. Models are usually grouped by their level of abstraction:

- **High-level / conceptual models** — close to how users think: the **E-R model**, the **Enhanced E-R (EER)** model, and object models. Used for design.
- **Representational / implementation models** — the **relational**, **network** and **hierarchical** models. Used by real DBMSs.
- **Low-level / physical models** — describe how data is stored (records, pointers, indexes).

The historical progression — know all five

Model	Structure	Relationships	Access	Example / status
Hierarchical	Tree — each record has one parent	1:N only , via parent-child links (pointers)	Navigate down from the root	IBM IMS . Fast for 1:N; cannot express M:N naturally; redundancy
Network	Graph — a record may have many parents	1:N and M:N via "sets" (owner-member), using pointers	Navigate along pointer chains	CODASYL/DBTG , IDMS. More flexible than hierarchical but complex ; explicitly named in your CS Degree syllabus (§17)

Relational	Tables (relations) of rows and columns	Represented by shared values (foreign keys) , not pointers	Declarative — SQL; the DBMS finds the data	Oracle, MySQL, PostgreSQL, SQL Server. The dominant model — E. F. Codd, 1970
Object-oriented (OODBMS)	Objects with attributes and methods; inheritance	Object references	Object query language	ObjectDB, db4o. Good for complex data (CAD, multimedia)
Object-relational (ORDBMS)	Relational tables extended with objects, user-defined types, inheritance	FKs + object references	SQL + object extensions	PostgreSQL, Oracle. The practical hybrid — named in your Forensic syllabus

The key contrast to state: hierarchical and network models connect records with **physical pointers**, so navigation is **procedural** (the programmer states *how* to reach the data). The relational model connects records by **matching data values** (foreign keys), so querying is **declarative** (state *what* you want; the DBMS decides how). This is why the relational model won: **physical data independence** and easy ad-hoc queries.

Also mention the **NoSQL** families (document, key-value, column-family, graph) as the modern answer to web-scale, schema-flexible data — but the exam is overwhelmingly relational.

Likely exam questions

- **[5]** What is a data model? Classify data models.
- **[5]** Differentiate between the hierarchical and network data models.
- **[5]** Why did the relational model replace the network and hierarchical models?
- **[10]** Explain the different types of data model with their structures, advantages and disadvantages.

MCQ traps

Trap	Correct answer
"Which model uses a tree?"	Hierarchical .
"Which model allows a record many parents?"	Network (graph).
"Who proposed the relational model, when?"	E. F. Codd, 1970 .
"Relational links records by?"	Shared values (foreign keys) , not pointers.
"CODASYL / DBTG refers to the?"	Network model.
"Tables extended with user-defined types?"	Object-relational model.

4. The Entity-Relationship (E-R) model

Concept

The **E-R model** (Peter Chen, 1976) is a **high-level conceptual** model used to design a database *before* choosing tables. It views the world as **entities** (things) linked by **relationships**, each described by **attributes**. The output is an **E-R diagram** — the single most drawable, most examinable artefact in the whole DBMS syllabus. Learn the notation until you can draw it fast.

The building blocks and their symbols

Concept	Symbol (hand-drawn)	Meaning
Entity type	Rectangle	A class of real-world objects: <code>STUDENT</code> , <code>COURSE</code>
Weak entity type	Double rectangle	An entity that cannot be identified by its own attributes alone
Attribute	Ellipse/oval	A property of an entity or relationship: <code>name</code> , <code>age</code>
Key attribute	Ellipse with the name underlined	Uniquely identifies an entity: <code>roll_no</code>
Relationship type	Diamond	An association among entities: <code>ENROLLS</code>
Identifying relationship	Double diamond	Links a weak entity to its owner
Multivalued attribute	Double ellipse	Can hold several values: <code>phone_numbers</code>
Composite attribute	Ellipse with child ellipses	Divisible into parts: <code>address</code> → street, city, pin
Derived attribute	Dashed ellipse	Computed, not stored: <code>age</code> (from <code>dob</code>)
Partial key (of a weak entity)	Dashed underline	Distinguishes weak entities within one owner

Attribute types — a favourite short question

Type	Meaning	Example
Simple (atomic)	Cannot be divided	<code>age</code> , <code>roll_no</code>
Composite	Made of sub-parts	<code>name</code> = first + last; <code>address</code> = street + city + pin
Single-valued	One value per entity	<code>date_of_birth</code>
Multivalued	Several values per entity	<code>phone</code> , <code>email</code>
Stored	Physically stored	<code>date_of_birth</code>
Derived	Computed from others	<code>age</code> from <code>date_of_birth</code>
Key	Uniquely identifies	<code>roll_no</code>
NULL	Value unknown or not applicable	<code>middle_name</code>

Relationships — degree, cardinality, participation

Degree = number of entity types participating in a relationship:

- **Unary (recursive)** — degree 1: `EMPLOYEE manages EMPLOYEE`.
- **Binary** — degree 2 (by far the commonest): `STUDENT enrolls COURSE`.
- **Ternary** — degree 3: `SUPPLIER supplies PART to PROJECT`.

Cardinality ratio = the maximum number of entities one entity can be related to:

Ratio	Meaning	Example
1:1 (one-to-one)	Each A relates to at most one B and vice-versa	<code>EMPLOYEE</code> — <code>manages</code> — <code>DEPARTMENT</code> (one manager per dept)

1:N (one-to-many)	One A relates to many B, each B to one A	DEPARTMENT — has — EMPLOYEE
M:N (many-to-many)	Many A to many B	STUDENT — enrolls — COURSE

Participation constraint = the *minimum* — must every entity take part?

- **Total participation** (existence dependency): drawn as a **double line** — *every* entity of the type must participate. E.g. every **LOAN** must belong to some **CUSTOMER**.
- **Partial participation: single line** — participation is optional. E.g. not every **CUSTOMER** has a **LOAN**.

Cardinality and participation together are the **(min, max)** notation: **(1,1)**, **(0,N)**, etc. Stating both the ratio *and* the participation is what separates a full answer from a partial one.

Weak entity types

A **weak entity** cannot be uniquely identified by its own attributes; it depends on a **strong (owner) entity** through an **identifying relationship**. It has only a **partial key (discriminator)** — unique only *within* one owner. Classic example: **DEPENDENT** of an **EMPLOYEE**. Two dependents of different employees may both be "Sunita, daughter"; a dependent's full key is **(employee_id, dependent_name)**. Draw the weak entity as a **double rectangle**, the identifying relationship as a **double diamond**, with **total participation** (double line) on the weak side, and the partial key **dashed-underlined**.

Enhanced E-R (EER): generalisation, specialisation, aggregation

These three extend the basic model and are explicitly listed in your task — expect a 10-marker.

Specialisation (top-down): start with a general entity (superclass) and split it into more specific subclasses that have extra attributes. **EMPLOYEE** → **ENGINEER**, **MANAGER**, **TECHNICIAN**. Each subclass **inherits** the superclass's attributes and relationships and adds its own.

Generalisation (bottom-up): the reverse — spot common features among several entities and combine them into a general superclass. **CAR** and **TRUCK** → **VEHICLE**. Generalisation and specialisation are the same "**is-a**" hierarchy approached from opposite directions.

Draw the **is-a** hierarchy as a superclass rectangle connected downward through a **triangle** (labelled "ISA") to its subclass rectangles.

Two constraint dimensions to mention:

- **Disjoint (d)** vs **overlapping (o)**: can an entity belong to *only one* subclass, or several? (An **EMPLOYEE** is either salaried or hourly = disjoint; a **PERSON** may be both **STUDENT** and **EMPLOYEE** = overlapping.)
- **Total** vs **partial** specialisation: must every superclass member belong to *some* subclass?

Aggregation: a way to express a **relationship of a relationship** — to treat a whole relationship set as a single higher-level entity so that another relationship can connect to it. Classic example: **EMPLOYEE works-on PROJECT** (a relationship); a **MANAGER supervises** that whole **works-on** combination. You aggregate **works-on** into an abstract entity and draw the **supervises** relationship to the **box drawn around** the aggregated relationship. It solves the problem that basic E-R cannot draw a relationship connected to another relationship.

Abstraction	Direction / idea	Keyword	Example
-------------	------------------	---------	---------

Specialisation	Top-down: split into subclasses	is-a	EMPLOYEE → MANAGER, ENGINEER
Generalisation	Bottom-up: combine into superclass	is-a	CAR, TRUCK → VEHICLE
Aggregation	Treat a relationship as an entity	part-of / has-a	Manager supervises (Employee works-on Project)

Worked example — a small E-R diagram to describe

Design an E-R schema for a college. Entities and their key attributes:

- **STUDENT** (roll_no, name[composite], dob, age[derived], phone[multivalued])
- **COURSE** (course_id, title, credits)
- **INSTRUCTOR** (emp_id, name, salary)
- **DEPENDENT** (name[partial key], relation) — **weak**, owned by **INSTRUCTOR**

Relationships:

- **ENROLLS** between STUDENT and COURSE — **M:N**, with attribute grade (a relationship attribute — an M:N relationship's own attribute, e.g. the grade a student earns in a course).
- **TEACHES** between INSTRUCTOR and COURSE — **1:N** (one instructor teaches many courses).
- **HAS_DEPENDENT** — identifying relationship (double diamond) between INSTRUCTOR and the weak entity DEPENDENT, total participation on DEPENDENT.

Describe it in words for the examiner, then draw: rectangles for the four entities, a double rectangle for DEPENDENT, diamonds for the relationships, ellipses for attributes with roll_no, course_id, emp_id underlined, age dashed, phone a double ellipse, name composite, and grade hanging off the ENROLLS diamond.

Likely exam questions

- **[5]** Define entity, attribute and relationship. List the different types of attribute.
- **[5]** Explain the different types of attribute with examples and their E-R symbols.
- **[5]** What is a weak entity? How is it represented? Give an example.
- **[5]** Differentiate between generalisation, specialisation and aggregation.
- **[5]** Explain cardinality ratios and participation constraints with examples.
- **[10]** Explain the E-R model. Describe all the symbols used in an E-R diagram with examples.
- **[15]** Draw an E-R diagram for a [college / hospital / bank] system. Show entities, attributes (including composite, multivalued, derived and key), relationships with their cardinality and participation, at least one weak entity, and one generalisation hierarchy.

MCQ traps

Trap	Correct answer
"An entity set is drawn as a?"	Rectangle . Relationship = diamond , attribute = ellipse .
"A derived attribute is drawn as?"	Dashed ellipse . Multivalued = double ellipse .
"A weak entity is drawn as?"	Double rectangle ; identifying relationship = double diamond .
"Total participation is shown by?"	A double line .
"Degree of a relationship means?"	Number of entity types in it.
"Combining lower entities into a higher one is?"	Generalisation (bottom-up).

"Relationship treated as an entity is?"	Aggregation .
"'is-a' relationship denotes?"	Generalisation/specialisation (inheritance).
"Who proposed the E-R model?"	Peter Chen (1976) .

5. Converting an E-R diagram to a relational schema

Concept

Design in E-R (easy to think in), then **mechanically translate** to tables (easy to implement).
 There are **seven standard rules**; knowing all seven and applying them to a given diagram is a classic 10/15-mark question named in *all three* of your syllabi.

#	E-R construct	Relational rule
1	Strong (regular) entity	Create a table with all its simple attributes; its key becomes the primary key
2	Weak entity	Create a table with its own attributes plus the primary key of the owner as a foreign key; the primary key = owner's PK + the partial key
3	1:1 relationship	Add the PK of one side into the other as a foreign key (put it on the side with total participation to avoid NULLs). No new table needed
4	1:N relationship	Add the PK of the " 1 " side into the " N " side table as a foreign key. No new table
5	M:N relationship	Create a new (junction/bridge) table containing the PKs of both entities as foreign keys; their combination is the primary key , plus any relationship attributes
6	Multivalued attribute	Create a separate table holding the attribute + the owner's PK; PK = both together
7	Composite attribute	Store only its simple components as separate columns; drop the composite name itself

For **generalisation/specialisation**, three strategies: (a) one table per subclass with the inherited attributes duplicated; (b) one table for the superclass and one per subclass sharing the key; (c) a single table for the whole hierarchy with a type-discriminator column and NULLs for inapplicable attributes.

Worked example

Take the college E-R model from §4 and translate it.

Rule 1 — strong entities:

```
STUDENT(roll_no PK, first_name, last_name, dob)      -- age is derived → not stored
COURSE(course_id PK, title, credits)
INSTRUCTOR(emp_id PK, name, salary)
```

name was composite → split into **first_name**, **last_name** (Rule 7). **age** was derived → not stored (compute on demand).

Rule 6 — multivalued attribute **phone** of STUDENT:

```
STUDENT_PHONE(roll_no FK, phone)    PK = (roll_no, phone)
```

Rule 4 — 1:N relationship **TEACHES** (one instructor, many courses): put the instructor's key

into COURSE:

```
COURSE(course_id PK, title, credits, emp_id FK → INSTRUCTOR)
```

Rule 5 — M:N relationship ENROLLS with attribute **grade**: make a junction table:

```
ENROLLS(roll_no FK → STUDENT, course_id FK → COURSE, grade)    PK = (roll_no, course_id)
```

Rule 2 — weak entity DEPENDENT owned by INSTRUCTOR:

```
DEPENDENT(emp_id FK → INSTRUCTOR, dep_name, relation)    PK = (emp_id, dep_name)
```

Final relational schema:

```
STUDENT(roll_no, first_name, last_name, dob)
STUDENT_PHONE(roll_no, phone)
COURSE(course_id, title, credits, emp_id)
INSTRUCTOR(emp_id, name, salary)
ENROLLS(roll_no, course_id, grade)
DEPENDENT(emp_id, dep_name, relation)
```

Note how the M:N **ENROLLS** became its own table (Rule 5) while the 1:N **TEACHES** needed only a foreign column (Rule 4) — **that difference is the most examined point in the whole topic.**

Likely exam questions

- **[5]** State the rules for converting an E-R diagram into relational tables.
- **[5]** How are 1:N and M:N relationships mapped to tables? Why do they differ?
- **[10]** Given an E-R diagram, convert it into a set of relational schemas, showing primary and foreign keys, and justify each step.

MCQ traps

Trap	Correct answer
"An M:N relationship becomes?"	A separate (junction) table with both PKs.
"A 1:N relationship is mapped by?"	Putting the " 1 "-side PK as an FK in the "N "-side table.
"A multivalued attribute becomes?"	A separate table .
"A weak entity's PK is?"	Owner's PK + its own partial key .
"A composite attribute is stored as?"	Its individual simple components .

6. The relational model — terminology

Concept

A **relation** is a table; the model rests on a handful of precise terms that examiners love to test because each maps to a plain word.

Formal term	Informal term	Meaning
Relation	Table	A set of tuples over the same attributes

Tuple	Row / record	One entity's data
Attribute	Column / field	A named property
Domain	—	The set of allowed atomic values for an attribute (e.g. <code>age</code> : integers 0–150)
Degree (arity)	—	The number of attributes (columns)
Cardinality	—	The number of tuples (rows)
Relation schema	Table structure	<code>STUDENT(roll_no, name, age)</code> — name + attributes
Relation instance	Table contents	The actual set of rows at a moment

Do not confuse degree and cardinality: degree counts **columns**, cardinality counts **rows**. A table `STUDENT(roll_no, name, age, dept)` with 50 students has **degree 4** and **cardinality 50**.

Properties of a relation (Codd)

- Each **tuple is unique** — no duplicate rows (a relation is a *set*).
- **Tuple order is immaterial** — rows have no inherent sequence.
- **Attribute order is immaterial** — columns are identified by name, not position.
- Each **cell holds a single atomic value** — this is the **first normal form** requirement; no repeating groups or multivalued cells.
- All values in a column come from the **same domain**.
- **NULL** marks an unknown or inapplicable value.

Likely exam questions

- [5] Define relation, tuple, attribute, domain, degree and cardinality with an example.
- [5] State the properties of a relation.
- [5] Differentiate between degree and cardinality of a relation.

MCQ traps

Trap	Correct answer
"Number of columns is the?"	Degree . Number of rows = cardinality .
"A row is formally a?"	Tuple .
"The set of permitted values for an attribute?"	Domain .
"A relation is a set of?"	Tuples — hence no duplicate rows.
"Order of rows in a relation matters?"	No .
"Every cell must be?"	Atomic (single-valued) — 1NF.

7. Keys

Concept

A **key** is one or more attributes that let you **identify tuples**. Everything in

normalization and integrity builds on keys, so get the hierarchy exact. Work from a single running example throughout:

```
STUDENT(roll_no, aadhaar, email, name, dept)
-- roll_no unique, aadhaar unique, email unique
```

Key	Definition	In the example
Super key	Any set of attributes that uniquely identifies a tuple (may have extras)	<code>{roll_no}</code> , <code>{roll_no, name}</code> , <code>{aadhaar}</code> , <code>{roll_no, aadhaar}</code> ... many
Candidate key	A minimal super key — no attribute can be removed and still stay unique	<code>{roll_no}</code> , <code>{aadhaar}</code> , <code>{email}</code>
Primary key	The one candidate key chosen to identify tuples; cannot be NULL, must be unique	<code>roll_no</code> (chosen)
Alternate key	Candidate keys not chosen as primary	<code>aadhaar</code> , <code>email</code>
Composite (concatenated) key	A key made of two or more attributes together	In <code>ENROLLS(roll_no, course_id, grade): (roll_no, course_id)</code>
Foreign key	An attribute in one relation that refers to the primary key of another (may repeat, may be NULL)	<code>emp_id</code> in <code>COURSE</code> referencing <code>INSTRUCTOR</code>
Surrogate key	A system-generated artificial key (auto-increment/UUID) with no business meaning	An added <code>student_id INT AUTO_INCREMENT</code>

The containment chain to state: every **primary key** and every **alternate key** is a **candidate key**; every candidate key is a **super key**. So `super key  $\supseteq$  candidate key  $\supseteq$  {primary key, alternate keys}`. A super key with any extra attribute removed that is still unique was not minimal — minimality is exactly what turns a super key into a candidate key.

Foreign key rules (referential integrity): a foreign-key value must either **match an existing primary-key value** in the referenced table **or be NULL**. On delete/update of the parent, the DBMS applies a chosen action: **CASCADE, SET NULL, SET DEFAULT, RESTRICT/NO ACTION**.

Likely exam questions

- **[5]** Define super key, candidate key, primary key and alternate key with one example.
- **[5]** What is a foreign key? How does it enforce referential integrity?
- **[5]** Differentiate between a primary key and a candidate key.
- **[10]** Explain the various types of keys in the relational model with a single worked example, and show the relationship among them.

MCQ traps

Trap	Correct answer
"A minimal super key is a?"	Candidate key.
"Candidate keys not chosen as primary are?"	Alternate keys.
"Which key cannot be NULL?"	Primary key (entity integrity).

"Which key can repeat and be NULL?"	Foreign key .
"A key of two or more attributes is?"	Composite key .
"Every candidate key is a?"	Super key (the reverse is not true).
"Auto-generated meaningless key?"	Surrogate key .

8. Integrity constraints

Concept

Integrity constraints are rules the data must always satisfy; the DBMS **rejects** any operation that would break one, which is how it keeps data valid without relying on application code.

Constraint	Rule	Example / violation
Domain constraint	Each attribute value must come from its declared domain/type	<code>age</code> must be an integer 0–150; <code>'abc'</code> is rejected
Not-NULL constraint	The attribute may not be empty	<code>name NOT NULL</code>
Key (uniqueness) constraint	No two tuples may have the same value of a candidate key	<code>UNIQUE(email)</code>
Entity integrity	The primary key cannot be NULL (and must be unique)	A row with <code>roll_no = NULL</code> is rejected
Referential integrity	A foreign key must match an existing PK value or be NULL	Inserting an order for a non-existent customer is rejected
Check / semantic constraint	An arbitrary boolean rule on values	<code>CHECK (salary > 0), CHECK (grade IN ('A', 'B', 'C', 'F'))</code>

Entity integrity and **referential integrity** are the two "big" relational constraints — name them first. Entity integrity guarantees every row is identifiable; referential integrity guarantees relationships between tables stay consistent (no "dangling" references).

Likely exam questions

- **[5]** What are integrity constraints? Explain entity integrity and referential integrity.
- **[5]** Explain domain, key and referential integrity constraints with examples.
- **[10]** Explain the various integrity constraints enforced by a relational DBMS, and how referential integrity is maintained on delete and update.

MCQ traps

Trap	Correct answer
"Primary key cannot be NULL is which rule?"	Entity integrity .
"A foreign key must match a PK or be NULL is?"	Referential integrity .
" <code>CHECK (salary>0)</code> is a?"	Check / domain (semantic) constraint .

"Deleting a parent row with **ON DELETE CASCADE** does?"

Deletes the matching child rows too.

9. Relational algebra

Concept

Relational algebra is the **procedural** query language underlying SQL: a set of operators that each take one or two relations and produce a new relation, so operations **compose**. (SQL is the practical language; relational *calculus* is the non-procedural theoretical counterpart.)

Learn the six **fundamental** operators plus the derived ones, each with a symbol and a one-line query.

Running tables:

```
EMP(eid, name, dept, salary)
DEPT(dept, location)
```

Operator	Symbol	Meaning	Example	Reads as
Select	σ (sigma)	Choose rows matching a condition	$\sigma_{\text{salary}>50000}$ (EMP)	"employees earning > 50000"
Project	π (pi)	Choose columns ; removes duplicates	$\pi_{\text{name}, \text{salary}}$ (EMP)	"names and salaries"
Union	$\cup$	Rows in either relation (union-compatible)	$\pi_{\text{name}}(A) \cup \pi_{\text{name}}(B)$	"names in A or B"
Set difference	$-$	Rows in first but not second	$\pi_{\text{eid}}(\text{EMP}) - \pi_{\text{eid}}(\text{MANAGERS})$	"employees who are not managers"
Cartesian product	$\times$	Every row of A paired with every row of B	$\text{EMP} \times \text{DEPT}$	all combinations
Rename	ρ (rho)	Rename a relation/attributes	$\rho_S(\text{EMP})$	needed for self-joins
Intersection	$\cap$	Rows in both (derived: $A - (A - B)$)	$\pi_{\text{name}}(A) \cap \pi_{\text{name}}(B)$	"names in both"
Natural join	$\bowtie$	Product + select on equal common attributes + drop the duplicate column	$\text{EMP} \bowtie \text{DEPT}$	employees with their department location
Theta / equi-join	$\bowtie_{\theta}$	Join on an explicit condition θ	$\text{EMP} \bowtie_{(\text{EMP.dept}=\text{DEPT.dept})} \text{DEPT}$	join on a stated predicate
Division	$\div$	Tuples of A associated with all of B	<i>"suppliers who supply every part"</i>	universal-quantifier queries

Union-compatibility (required for $\cup$, $\cap$, $-$): both relations must have the **same number of attributes** and **matching domains**. This is a standard MCQ.

Worked queries

"Find the names of employees in the 'Sales' department earning more than 40000."

```
π_name ( σ_(dept='Sales' ∧ salary>40000) (EMP) )
```

Selection filters rows first, projection keeps the **name** column.

"Find the locations of departments that employ 'Ravi'."

```
π_location ( σ_(name='Ravi') (EMP ⋈ DEPT) )
```

"Employees who work in a department located in 'Kohima'."

```
π_name ( EMP ⋈ (σ_location='Kohima' (DEPT)) )
```

Order matters for efficiency: apply **selection and projection as early as possible** (push them below the join) so the join works on fewer, narrower rows. This is the core idea of **query optimisation** (named in the Diploma syllabus).

Likely exam questions

- **[5]** List the fundamental operations of relational algebra with their symbols.
- **[5]** Differentiate between the SELECT (σ) and PROJECT (π) operations.
- **[5]** What is union-compatibility? Which operations require it?
- **[5]** Explain the natural join with an example.
- **[10]** Explain relational algebra operations with examples, and write relational-algebra expressions for a given set of queries.

MCQ traps

Trap	Correct answer
" σ (sigma) selects?"	Rows (tuples). π (pi) selects columns .
"Which operation removes duplicate rows?"	Projection (π).
"Union requires the two relations to be?"	Union-compatible .
"Natural join combines rows on?"	Equal values of common attributes .
"Which operator answers 'for all'?"	Division ($\div$).
"Relational algebra is procedural or declarative?"	Procedural ; relational <i>calculus</i> is declarative.

10. Normalization

This is the highest-scoring theory-plus-numerical topic in the DBMS paper. Learn to work a decomposition by hand at every normal form.

Concept

Normalization is the process of organising attributes into relations to **reduce redundancy and eliminate update anomalies**, by successively applying stricter rules ("normal forms"). The enemy it fights is the **anomaly**:

- **Insertion anomaly** — you cannot add a fact without inventing unrelated data (can't record a new course with no student enrolled).

- **Update anomaly** — one fact stored many times; you must update every copy or risk inconsistency (change an instructor's phone in 40 rows).
- **Deletion anomaly** — removing one fact accidentally destroys another (deleting the last student on a course loses the course itself).

All three come from **redundancy**, and redundancy comes from **functional dependencies** that do not centre on a key. Fix the dependencies and the anomalies vanish.

Functional dependency (FD)

An FD $X \rightarrow Y$ ("X determines Y") means: whenever two tuples agree on X, they must agree on Y. X is the **determinant**. Example: $\text{roll_no} \rightarrow \text{name}$ (a roll number determines exactly one name).

Types:

- **Trivial FD**: $Y \subseteq X$ (e.g. $\{\text{roll_no}, \text{name}\} \rightarrow \text{name}$) — always holds, uninteresting.
- **Non-trivial FD**: $Y \not\subseteq X$ — the useful kind.
- **Full FD**: Y depends on **the whole** of a composite X, not part of it.
- **Partial FD**: Y depends on **part** of a composite key — the 2NF villain.
- **Transitive FD**: $X \rightarrow Y$ and $Y \rightarrow Z$ give $X \rightarrow Z$ where Y is non-key — the 3NF villain.

Armstrong's axioms

The **sound and complete** inference rules for deriving all FDs implied by a set. The three primary axioms:

Axiom	Rule
Reflexivity	If $Y \subseteq X$ then $X \rightarrow Y$ (trivial dependencies)
Augmentation	If $X \rightarrow Y$ then $XZ \rightarrow YZ$ for any Z
Transitivity	If $X \rightarrow Y$ and $Y \rightarrow Z$ then $X \rightarrow Z$

Derived (secondary) rules — handy shortcuts:

Rule	Statement
Union	If $X \rightarrow Y$ and $X \rightarrow Z$ then $X \rightarrow YZ$
Decomposition	If $X \rightarrow YZ$ then $X \rightarrow Y$ and $X \rightarrow Z$
Pseudo-transitivity	If $X \rightarrow Y$ and $WY \rightarrow Z$ then $WX \rightarrow Z$

Sound = derives only true FDs; **complete** = derives *all* true FDs. That phrase earns a mark.

Attribute closure (X^+)

X^+ is the set of **all attributes functionally determined by X**. Algorithm: start with $X^+ = X$; repeatedly, for every FD $A \rightarrow B$ where $A \subseteq X^+$, add B to X^+ ; stop when nothing new is added.

Two uses:

1. **Is X a super key?** X is a super key **iff** $X^+ = \text{all attributes}$.
2. **Does $X \rightarrow Y$ hold?** Yes iff $Y \subseteq X^+$.

Worked closure. $R(A,B,C,D,E)$, FDs: $A \rightarrow B$, $B \rightarrow C$, $CD \rightarrow E$.

- $\{A\}^+$: start $\{A\}$; $A \rightarrow B$ adds B $\rightarrow \{A,B\}$; $B \rightarrow C$ adds C $\rightarrow \{A,B,C\}$; $CD \rightarrow E$ needs D (absent) $\rightarrow$ stop.

$\{A\}^+ = \{A,B,C\}$. Not a super key (D, E missing).

- $\{A,D\}^+$: $\{A,D\} \rightarrow +B \rightarrow +C$ (now $\{A,B,C,D\}$) $\rightarrow CD \rightarrow E$ adds E $\rightarrow \{A,B,C,D,E\} = \text{all}$. So **AD is a**

super key, and since neither A nor D alone gives everything, **AD is a candidate key**.

Finding candidate keys — the reliable method

1. Find attributes that appear **only on the left** of FDs (or in no FD): they **must** be in every candidate key.
2. Attributes appearing **only on the right** are **never** in a candidate key.
3. Take the closure of the "must-be-in" set. If it is all attributes, that is the key. Otherwise add other attributes one at a time and test closure, keeping the sets **minimal**.

Worked key-finding. R(A,B,C,D), FDs: $A \rightarrow B$, $B \rightarrow C$, $C \rightarrow D$.

A appears only on the left $\rightarrow$ A is essential. $\{A\}^+ = \{A,B,C,D\} = \text{all}$. **A is the only candidate key**. B, C, D are prime nowhere else, so no other candidate key exists.

The normal forms — each with a worked decomposition

First Normal Form (1NF)

Rule: every cell holds a **single atomic value** — no repeating groups, no multivalued attributes, no nested tables.

Unnormalised (0NF):

```
STUDENT(roll_no, name, phones)
(1, Ravi, "9000,9001")      ← two phones in one cell
```

1NF — atomise (one row per value):

```
STUDENT(roll_no, name, phone)
(1, Ravi, 9000)
(1, Ravi, 9001)
```

The table is now 1NF but has introduced redundancy (**name** repeats), which the higher forms fix.

Second Normal Form (2NF)

Rule: 1NF and no **partial dependency** — every non-prime attribute is **fully** functionally dependent on **the whole** of every candidate key. (Only relevant when the key is **composite**; a single-attribute key is automatically in 2NF.)

Violating example — a store's line-item table:

```
R(order_id, product_id, qty, product_name, product_price)
Key = (order_id, product_id)
FDs: (order_id, product_id) → qty
      product_id → product_name, product_price    ← PARTIAL (depends on part of the key)
```

product_name depends only on **product_id**, not the whole key. **Decompose:**

```
ORDER_ITEM(order_id, product_id, qty)      -- full dependency only
PRODUCT(product_id, product_name, product_price)
```

Third Normal Form (3NF)

Rule: 2NF and no **transitive dependency** of a non-prime attribute on a candidate key. Equivalently, for **every** non-trivial FD $X \rightarrow Y$, **either X is a super key, or Y is a prime attribute** (part of some candidate key).

Violating example:

```
STUDENT(roll_no, name, dept_id, dept_head)
Key = roll_no
FDs: roll_no → dept_id , dept_id → dept_head
    ⇒ roll_no → dept_head    TRANSITIVELY    (dept_id is non-key)
```

Decompose to remove the transitive dependency:

```
STUDENT(roll_no, name, dept_id)
DEPT(dept_id, dept_head)
```

Now `dept_head` is stored **once per department**, not once per student — the update anomaly is gone.

Boyce-Codd Normal Form (BCNF)

Rule: for **every** non-trivial FD $X \rightarrow Y$, **X must be a super key**. BCNF is stricter than 3NF; the only difference is that 3NF forgives an FD whose right side is a *prime* attribute, BCNF does not. So **3NF $\subset$ BCNF may fail only when there are overlapping candidate keys**.

Classic violating example:

```
R(student, subject, teacher)
Rules: each teacher teaches exactly ONE subject    → teacher → subject
      each (student, subject) has one teacher      → (student, subject) → teacher
Candidate keys: (student, subject) and (student, teacher)
```

`teacher → subject` has a determinant (`teacher`) that is **not a super key** → violates BCNF (but the table *is* in 3NF, because `subject` is a prime attribute). **Decompose:**

```
R1(teacher, subject)      -- teacher → subject
R2(student, teacher)     -- who is taught by whom
```

The 3NF/BCNF trade-off to state: decomposition into BCNF is always **lossless**, but may **not preserve all dependencies**; 3NF can always be achieved **losslessly and dependency-preserving**. That is precisely why 3NF is the practical target and BCNF the ideal.

Fourth Normal Form (4NF)

Rule: BCNF and no non-trivial multivalued dependency (MVD) $X \twoheadrightarrow Y$ unless X is a super key. An **MVD** $X \twoheadrightarrow Y$ means X determines a *set* of Y values **independently** of the other attributes. It appears when one table crams **two independent multivalued facts** together.

Violating example — a person's skills and languages are independent:

```
R(emp, skill, language)
emp → skill and emp → language    (independent)
(1, Java, English)
(1, Java, Hindi)
(1, Python, English)
(1, Python, Hindi)    ← every skill × every language must be listed: redundancy
```

Decompose into two tables, one per independent fact:

```
EMP_SKILL(emp, skill)
EMP_LANG(emp, language)
```

(Beyond 4NF: **5NF/PJNF** handles join dependencies; rarely examined — mention it exists.)

Summary ladder

Form	Removes	One-line rule
1NF	Repeating groups / multivalued cells	All values atomic
2NF	Partial dependencies	No non-prime attribute depends on part of a composite key
3NF	Transitive dependencies	Every determinant is a super key , <i>or</i> the dependent is prime
BCNF	Remaining anomalies from overlapping keys	Every determinant is a super key
4NF	Multivalued dependencies	No non-trivial MVD unless the determinant is a super key

Lossless join and dependency preservation

When you split R into R1 and R2, two properties matter:

Lossless (non-additive) join: re-joining R1 and R2 must give back **exactly** R — no spurious

extra rows. **Test:** the decomposition of R into R1, R2 is lossless **iff** the common

attributes **(R1 ∩ R2)** form a **super key of R1 or of R2** — i.e. $(R1 \cap R2) \rightarrow R1$ or

$(R1 \cap R2) \rightarrow R2$. A lossy decomposition invents rows that were never there and is unacceptable.

Worked test. R(A,B,C), FDs $A \rightarrow B$, $B \rightarrow C$. Split into R1(A,B) and R2(B,C). Common attribute = {B}.

Is B a key of R2? $B \rightarrow C$ so $\{B\}^+ \supseteq \{B,C\}$ = all of R2 → **yes**. Decomposition is **lossless**. ✓

Dependency preservation: every original FD should be enforceable by checking **one table**

alone, without a join. If $(F1 \cup F2)^+ = F^+$, dependencies are preserved. In the example above,

$A \rightarrow B$ lives in R1 and $B \rightarrow C$ lives in R2 — **both preserved**. ✓

Likely exam questions

- **[5]** Define functional dependency. Differentiate full, partial and transitive dependencies.
- **[5]** State Armstrong's axioms. What does it mean that they are sound and complete?
- **[5]** Compute the closure $\{A\}^+$ for the given FD set and state whether A is a candidate key.
- **[5]** Differentiate between 3NF and BCNF with an example.
- **[5]** What is a lossless-join decomposition? State the test.
- **[10]** Explain 1NF, 2NF and 3NF with a single running example, decomposing at each step.
- **[15]** What is normalization? Explain 1NF to BCNF with worked decompositions, and discuss lossless join and dependency preservation, stating why 3NF is usually preferred to BCNF.

MCQ traps

Trap	Correct answer
"1NF eliminates?"	Repeating groups / multivalued attributes .
"2NF removes?"	Partial dependency; needs a composite key to be at risk.
"3NF removes?"	Transitive dependency.
"BCNF requires every determinant to be a?"	Super key .
"4NF deals with?"	Multivalued dependencies (MVD) .

"Which is stronger, 3NF or BCNF?"	BCNF.
"3NF vs BCNF — which guarantees dependency preservation?"	3NF.
"Armstrong's axioms are?"	Sound and complete.
"X ⁺ = all attributes means X is a?"	Super key.
"A decomposition is lossless if the common attributes are a?"	Super key of one of the tables.

11. SQL — Structured Query Language

Concept

SQL is the standard **declarative** language for relational databases — you state *what* you want, the DBMS decides *how*. It is (mostly) **case-insensitive** for keywords, ends statements with `;`, and is grouped into sub-languages by purpose. Getting the four categories right is a guaranteed MCQ.

Sub-language	Purpose	Commands
DDL — Data Definition	Define/alter the structure (schema)	<code>CREATE</code> , <code>ALTER</code> , <code>DROP</code> , <code>TRUNCATE</code> , <code>RENAME</code>
DML — Data Manipulation	Work with the data (rows)	<code>SELECT</code> , <code>INSERT</code> , <code>UPDATE</code> , <code>DELETE</code>
DCL — Data Control	Permissions	<code>GRANT</code> , <code>REVOKE</code>
TCL — Transaction Control	Transaction boundaries	<code>COMMIT</code> , <code>ROLLBACK</code> , <code>SAVEPOINT</code> , <code>SET TRANSACTION</code>

(Some texts count `SELECT` as a separate DQL — Data Query Language. `TRUNCATE`/`DROP` are DDL and are auto-committed, which is why they cannot be rolled back — a favourite trap.)

DDL by example

```
CREATE TABLE STUDENT (
  roll_no    INT          PRIMARY KEY,
  name      VARCHAR(50)  NOT NULL,
  dept      VARCHAR(20)  DEFAULT 'CSE',
  cgpa      DECIMAL(3,2) CHECK (cgpa BETWEEN 0 AND 10),
  dept_id   INT,
  FOREIGN KEY (dept_id) REFERENCES DEPT(dept_id) ON DELETE SET NULL
);

ALTER TABLE STUDENT ADD email VARCHAR(60) UNIQUE;  -- add a column
ALTER TABLE STUDENT DROP COLUMN email;             -- remove it
DROP TABLE STUDENT;                                -- delete the whole table + data + structure
TRUNCATE TABLE STUDENT;                             -- delete all rows, keep structure (fast, no rollback)
```

DELETE vs TRUNCATE vs DROP — a standard comparison:

	DELETE	TRUNCATE	DROP
--	--------	----------	------

Type	DML	DDL	DDL
Removes	Selected rows (with WHERE)	All rows	Table itself (structure + data)
WHERE clause	Yes	No	No
Rollback	Yes (logged)	Usually no (auto-commit)	No
Speed	Slow (row by row)	Fast	Fast
Resets identity	No	Yes	N/A

DML by example

```
INSERT INTO STUDENT (roll_no, name, dept) VALUES (1, 'Ravi', 'CSE');
UPDATE STUDENT SET cgpa = 8.5 WHERE roll_no = 1;
DELETE FROM STUDENT WHERE cgpa < 4.0;
SELECT name, cgpa FROM STUDENT WHERE dept = 'CSE' ORDER BY cgpa DESC;
```

The **SELECT** skeleton and its **logical evaluation order** (worth stating, because it explains why you cannot use a column alias or an aggregate in **WHERE**):

```
SELECT  [DISTINCT] columns / aggregates      -- 5
FROM    tables / joins                      -- 1
WHERE   row filter (before grouping)        -- 2
GROUP BY grouping columns                  -- 3
HAVING  group filter (after grouping)       -- 4
ORDER BY sort                             -- 6
LIMIT   n                                 -- 7
```

Joins — every type with example and output

Two tables:

EMP			DEPT	
eid	name	dept_id	dept_id	dname
1	Ravi	10	10	Sales
2	Sita	20	20	IT
3	Anil	NULL	30	HR

INNER JOIN — only matching rows from both:

```
SELECT e.name, d.dname
FROM EMP e INNER JOIN DEPT d ON e.dept_id = d.dept_id;
```

name	dname
Ravi	Sales
Sita	IT

(Anil dropped — no match; HR dropped — no employee.)

LEFT (OUTER) JOIN — all left rows, NULLs where no right match:

```
SELECT e.name, d.dname FROM EMP e LEFT JOIN DEPT d ON e.dept_id = d.dept_id;
```

name	dname
Ravi	Sales
Sita	IT
Anil	NULL

RIGHT (OUTER) JOIN — all right rows, NULLs where no left match:

```
| name | dname |
|---|---|
| Ravi | Sales |
| Sita | IT |
| NULL | HR |
```

FULL OUTER JOIN — all rows from both sides, NULLs where unmatched:

```
| name | dname |
|---|---|
| Ravi | Sales |
| Sita | IT |
| Anil | NULL |
| NULL | HR |
```

CROSS JOIN — Cartesian product: every EMP row × every DEPT row (3 × 3 = 9 rows).

SELF JOIN — a table joined to itself (manager/employee):

```
SELECT e.name AS emp, m.name AS manager
FROM EMP e JOIN EMP m ON e.mgr_id = m.eid;
```

Join summary:

Join	Returns
INNER	Only rows matching in both
LEFT	All left + matching right (NULL if none)
RIGHT	All right + matching left (NULL if none)
FULL OUTER	All rows from both, matched where possible
CROSS	Cartesian product (all combinations)
SELF	A table joined to itself

Aggregates, GROUP BY and HAVING

Aggregate functions: **COUNT**, **SUM**, **AVG**, **MIN**, **MAX**. They collapse many rows into one value and **ignore NULLs** (except **COUNT (*)**, which counts rows).

```
SELECT dept_id, COUNT(*) AS headcount, AVG(salary) AS avg_sal
FROM EMP
WHERE salary IS NOT NULL           -- filter ROWS before grouping
GROUP BY dept_id
HAVING AVG(salary) > 50000         -- filter GROUPS after aggregating
ORDER BY avg_sal DESC;
```

WHERE vs HAVING — the single most examined SQL distinction: **WHERE filters individual rows before grouping** and **cannot contain aggregates**; **HAVING filters groups after aggregation** and **can**. If there is no **GROUP BY**, **HAVING** treats the whole result as one group.

Subqueries

A query nested inside another. Types:

- **Scalar** — returns one value: `WHERE salary > (SELECT AVG(salary) FROM EMP)`.
- **Multi-row** — with **IN, ANY, ALL**: `WHERE dept_id IN (SELECT dept_id FROM DEPT WHERE location='Kohima')`.
- **Correlated** — inner query references the outer row, re-evaluated per row:

```
SELECT name FROM EMP e
WHERE salary > (SELECT AVG(salary) FROM EMP WHERE dept_id = e.dept_id);
-- employees earning above their own department's average
```

- **EXISTS** — true if the subquery returns any row.

Views

A **view** is a **virtual table** defined by a stored query; it holds no data of its own but presents the result of its query on demand.

```
CREATE VIEW cse_toppers AS
SELECT roll_no, name, cgpa FROM STUDENT WHERE dept='CSE' AND cgpa > 8;
```

Uses: **security** (expose only some columns/rows), **simplicity** (hide complex joins), **logical data independence**. Limitations: a view is generally **updatable only if it maps to one base table without aggregates, DISTINCT, GROUP BY or joins**; complex views are read-only. A **materialised view** does store the result and must be refreshed.

Indexes

An **index** is an auxiliary structure (usually a **B+ tree**, sometimes a **hash**) that speeds up **retrieval** on the indexed column(s), at the cost of **extra storage** and **slower inserts/updates** (the index must be maintained).

```
CREATE INDEX idx_dept ON EMP(dept_id);
CREATE UNIQUE INDEX idx_email ON STUDENT(email);
```

A **primary/clustered index** determines the **physical order** of rows (one per table); a **secondary/non-clustered index** is a separate structure with pointers (many allowed). Rule of thumb: **index columns used in WHERE/JOIN/ORDER BY; do not over-index write-heavy tables**.

Stored procedures and functions

A **stored procedure** is a **named, precompiled block of SQL** stored in the database and invoked by name — reducing network traffic, centralising logic and improving security (grant **EXECUTE** instead of table access).

```
CREATE PROCEDURE raise_salary(IN p_eid INT, IN p_pct DECIMAL(4,2))
BEGIN
    UPDATE EMP SET salary = salary * (1 + p_pct/100) WHERE eid = p_eid;
END;
CALL raise_salary(1, 10); -- give employee 1 a 10% raise
```

A **function** is similar but **returns a value** and is usable **inside a query**; a procedure performs actions and is **called**. Both may take parameters (**IN**, **OUT**, **INOUT**).

Triggers

A **trigger** is a procedure that fires **automatically** in response to a data event (**INSERT**, **UPDATE**, **DELETE**) on a table — the database's event handler. Timing is **BEFORE** or **AFTER**, granularity is **FOR EACH ROW** (row-level) or statement-level.

```
CREATE TRIGGER audit_salary
AFTER UPDATE ON EMP
FOR EACH ROW
BEGIN
    INSERT INTO salary_log(eid, old_sal, new_sal, changed_at)
    VALUES (OLD.eid, OLD.salary, NEW.salary, NOW());
END;
```

OLD = the row before, **NEW** = the row after. Uses: **auditing**, enforcing complex business rules, maintaining derived/summary data, referential actions. Overuse hurts performance and hides logic — mention that as a disadvantage.

Likely exam questions

- **[5]** Differentiate between DDL, DML, DCL and TCL commands.
- **[5]** Differentiate between DELETE, TRUNCATE and DROP.
- **[5]** Differentiate between WHERE and HAVING clauses.
- **[5]** What is a view? State its advantages and when it is updatable.
- **[5]** What is a trigger? Explain BEFORE and AFTER triggers with an example.
- **[5]** Differentiate between a stored procedure and a function.
- **[10]** Explain all the join operations in SQL with example queries and their output.
- **[10]** Explain aggregate functions with GROUP BY and HAVING, and write SQL for a set of given queries.
- **[15]** Explain the categories of SQL. Write and explain queries demonstrating joins, subqueries, aggregates, views, indexes, a stored procedure and a trigger.

MCQ traps

Trap	Correct answer
" CREATE , ALTER , DROP belong to?"	DDL .
" GRANT / REVOKE are?"	DCL .
" COMMIT / ROLLBACK are?"	TCL .
"Which cannot be rolled back?"	TRUNCATE / DROP (DDL, auto-commit).
"WHERE can contain an aggregate?"	No — use HAVING .
"A view stores data?"	No (a materialised view does).
"INNER JOIN returns?"	Only matching rows from both tables.
"LEFT JOIN keeps?"	All left-table rows.

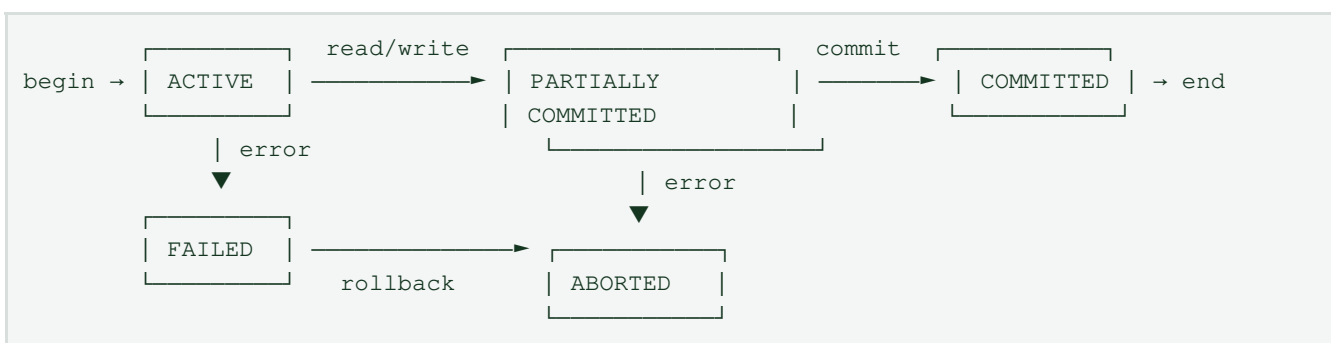
"Which fires automatically on INSERT/UPDATE/DELETE?"	A trigger .
"A function differs from a procedure by?"	It returns a value and is usable in a query.
"COUNT(*) vs COUNT(col)?"	<code>COUNT (*)</code> counts rows; <code>COUNT (col)</code> ignores NULLs .

12. Transactions and ACID

Concept

A **transaction** is a **logical unit of work** — a sequence of operations that must be treated as a **single indivisible action**: either **all** of it happens, or **none** of it does. The canonical example is a bank transfer of ₹100 from A to B: debit A, credit B. If the system crashes after the debit but before the credit, ₹100 vanishes. The transaction concept forbids that partial state.

Transaction states: *Active* → *Partially committed* (last operation done) → **Committed** (changes made permanent) — or from Active/Partially committed → *Failed* → **Aborted** (rolled back). Draw this as a small state diagram; it is often worth 3–4 marks.



The ACID properties — the core 5/10-marker

Property	Guarantee	Ensured by
Atomicity	All-or-nothing — a transaction's operations either all complete or none do	Recovery manager (undo via the log)
Consistency	A transaction takes the database from one valid state to another , preserving all constraints	Application logic + integrity constraints
Isolation	Concurrent transactions do not interfere; each behaves as if it ran alone	Concurrency control (locking, timestamps)
Durability	Once committed , changes survive any subsequent failure	Recovery manager (write-ahead log, flush to disk)

Mnemonic: **A**tomicity, **C**onsistency, **I**solation, **D**urability. Pair each with its enforcer — that pairing is what turns a definition into a full answer.

Concurrency problems (why isolation matters)

Without isolation, interleaved transactions cause anomalies:

Problem	What happens
Lost update	Two transactions read the same value and both write; the second overwrites the first's update — one update is lost
Dirty read (uncommitted dependency)	T2 reads a value T1 wrote, then T1 aborts — T2 used data that never officially existed
Unrepeatable read	T1 reads a row twice and gets different values because T2 updated it in between
Phantom read	T1 re-runs a range query and finds new rows inserted by T2

SQL isolation levels trade correctness for concurrency:

Level	Prevents
READ UNCOMMITTED	Nothing (allows dirty reads)
READ COMMITTED	Dirty reads
REPEATABLE READ	Dirty + unrepeatable reads
SERIALIZABLE	All, including phantoms — behaves as if transactions ran one at a time

Likely exam questions

- **[5]** What is a transaction? Explain its states with a diagram.
- **[5]** Explain the ACID properties of a transaction.
- **[5]** Explain the concurrency problems: lost update, dirty read and unrepeatable read.
- **[10]** What is a transaction? Explain the ACID properties, stating how each is enforced, and describe the transaction state diagram.

MCQ traps

Trap	Correct answer
"All-or-nothing' is which property?"	Atomicity .
"Committed changes survive a crash — which property?"	Durability .
"Isolation is ensured by?"	Concurrency control .
"Reading uncommitted data that is later rolled back is a?"	Dirty read .
"Highest isolation level?"	SERIALIZABLE .
"A transaction is a logical unit of?"	Work .

13. Schedules and serializability

Concept

A **schedule** is the chronological interleaving of the operations of several concurrent

transactions. A **serial schedule** runs transactions one after another (always correct but slow, no concurrency). We want the speed of interleaving with the correctness of serial execution — so we ask: is a given interleaved schedule **serializable**, i.e. **equivalent to some serial schedule**?

Conflict serializability is the practical test. Two operations **conflict** iff they belong to **different transactions**, access the **same data item**, and **at least one is a write**. The conflicting pairs are **R-W, W-R, W-W** (read-read never conflicts). A schedule is **conflict-serializable** if it can be transformed into a serial schedule by **swapping only non-conflicting adjacent operations**.

Precedence (serialization) graph — the algorithm to draw

1. Make a **node for each transaction**.
2. Draw an edge $T_i \rightarrow T_j$ whenever an operation of T_i **precedes and conflicts with** an operation of T_j (T_i reads/writes an item that T_j later writes, or T_i writes an item T_j later reads/writes).
3. **The schedule is conflict-serializable iff the graph has NO cycle**. A topological sort of an acyclic graph gives the equivalent serial order.

Worked example

Schedule S: R1 (A) W1 (A) R2 (A) W2 (A) R2 (B) W2 (B) R1 (B) W1 (B)

- W1 (A) then R2 (A)/W2 (A) on A $\rightarrow T1 \rightarrow T2$.
- W2 (B) then R1 (B)/W1 (B) on B $\rightarrow T2 \rightarrow T1$.

The graph has both $T1 \rightarrow T2$ and $T2 \rightarrow T1$ $\rightarrow$ a **cycle** $\rightarrow$ **not conflict-serializable**. If only the first conflict existed, the graph would be $T1 \rightarrow T2$ (acyclic) $\rightarrow$ serializable, equivalent to the serial order **T1, T2**.

View serializability is a weaker, more permissive notion (based on which transaction reads whose writes and who does the final write); every conflict-serializable schedule is view-serializable, but not vice versa. View-serializability testing is NP-hard, so **conflict serializability is what systems actually use**.

Recoverability is a separate concern: a schedule is **recoverable** if a transaction commits only *after* every transaction whose data it read has committed — otherwise an abort could force an already-committed transaction to be undone. **Cascadeless** schedules go further by allowing a read only of *committed* data, avoiding **cascading rollback**.

Likely exam questions

- [5] Define schedule, serial schedule and serializable schedule.
- [5] When do two operations conflict? List the conflicting pairs.
- [5] Explain conflict serializability using a precedence graph.
- [10] Given a schedule, draw its precedence graph and determine whether it is conflict-serializable; if so, give the equivalent serial schedule.

MCQ traps

Trap	Correct answer
------	----------------

"A schedule is conflict-serializable iff its precedence graph has?"	No cycle.
"Which pair does NOT conflict?"	Read-Read.
"Two ops conflict if same item, different txns and?"	At least one is a write.
"Every conflict-serializable schedule is also?"	View-serializable.
"Avoiding cascading rollback needs?"	Cascadeless schedule.

14. Concurrency control

Concept

Concurrency control is the mechanism that **produces only serializable, recoverable schedules** while still allowing interleaving. The two main families are **locking** (pessimistic) and **timestamp ordering** (optimistic about order).

Lock-based protocols

A **lock** is a privilege a transaction acquires on a data item before accessing it.

Lock mode	Grants	Compatible with
Shared (S) — read lock	Reading	Other S locks (many readers)
Exclusive (X) — write lock	Reading and writing	Nothing — one writer, no readers

Lock compatibility matrix: S–S compatible; S–X, X–S, X–X **incompatible**. Simply locking is not enough to guarantee serializability — *when* you release locks matters, which is why we need 2PL.

Two-Phase Locking (2PL) — the central protocol

Rule: every transaction obtains all its locks **before releasing any** — so its lifetime splits into two phases:

1. **Growing phase** — locks are **acquired**, none released.
2. **Shrinking phase** — locks are **released**, none acquired.

The moment of the first release is the **lock point**. **2PL guarantees conflict serializability**. Variants:

Variant	Rule	Solves
Basic 2PL	Growing then shrinking	Serializability, but risks deadlock and cascading rollback
Strict 2PL	Hold all exclusive (write) locks until commit/abort	Prevents cascading rollback ; recoverable
Rigorous 2PL	Hold all locks (S and X) until commit/abort	Simplest; serial order = commit order

Conservative (static) 2PL	Acquire all locks before starting	Deadlock-free (but needs to predeclare, low concurrency)
----------------------------------	---	---

Deadlock: two transactions each hold a lock the other needs, waiting forever. Handling:

- **Prevention** — timestamp schemes **wait-die** (older waits, younger dies/aborts) and **wound-wait** (older wounds/aborts younger, younger waits).
- **Avoidance** — conservative 2PL.
- **Detection** — build a **wait-for graph**; a **cycle = deadlock**; abort a victim.
- **Timeout** — abort a transaction that waits too long.

Timestamp ordering (TO)

Each transaction gets a unique **timestamp TS(T)** at start (older = smaller). Each data item keeps a **read-timestamp** and **write-timestamp**. The protocol allows an operation only if it respects timestamp order; otherwise the offending transaction is **aborted and restarted with a new timestamp**. It **orders transactions by their start time**, produces serializable schedules, and is **deadlock-free** (no waiting), but can cause **starvation** (a transaction repeatedly restarted). The **Thomas Write Rule** optimises it by ignoring obsolete writes.

Multiversion concurrency control (MVCC) keeps multiple versions so readers never block writers (used by PostgreSQL, Oracle); **optimistic concurrency control** validates only at commit, best when conflicts are rare.

	Locking (2PL)	Timestamp ordering
Style	Pessimistic — block until safe	Order by start time, abort if violated
Deadlock	Possible	Impossible (no waiting)
Starvation	Possible	Possible (repeated restarts)
Serial order	By lock point	By timestamp

Likely exam questions

- **[5]** What are shared and exclusive locks? Give the lock-compatibility matrix.
- **[5]** Explain the two phases of the 2PL protocol. What does it guarantee?
- **[5]** Differentiate between strict 2PL and rigorous 2PL.
- **[5]** What is a deadlock? Explain wait-die and wound-wait.
- **[10]** Explain lock-based concurrency control and the 2PL protocol with its variants, and discuss deadlock handling.
- **[10]** Compare lock-based and timestamp-based concurrency control.

MCQ traps

Trap	Correct answer
"2PL guarantees?"	Conflict serializability .
"In the growing phase a transaction may?"	Only acquire locks.
"Which lock is compatible with another shared lock?"	Shared (S) .

"Strict 2PL holds which locks till commit?"	Exclusive (write) locks.
"Which protocol is deadlock-free?"	Timestamp ordering (and conservative 2PL).
"Cycle in the wait-for graph means?"	Deadlock-
"Older transaction waits, younger aborts — this is?"	Wait-die-

15. Database recovery

Concept

Recovery restores the database to the **most recent consistent state** after a failure (transaction error, system crash, disk failure). Its job is to enforce **atomicity** (undo partial transactions) and **durability** (redo committed ones). The central tool is the **transaction log** written **before** the data — the **Write-Ahead Logging (WAL)** principle.

The WAL rule (state it): the **log record describing a change must reach stable storage before the changed data page does**, and all log records of a transaction must be flushed before it commits. Without WAL there would be nothing to undo/redo a crash with.

A log record looks like `<T, X, old_value, new_value>` (plus `<T start>`, `<T commit>`, `<T abort>`).

Log-based recovery techniques

Technique	When data is written to disk	On recovery
Deferred update (NO-UNDO/REDO)	Only after commit — changes buffered until then	REDO committed transactions; uncommitted ones never touched the DB, so no undo needed
Immediate update (UNDO/REDO)	Changes may be written before commit	UNDO uncommitted transactions (restore old values) and REDO committed ones (apply new values)

The rule for immediate update: **undo in reverse order, redo in forward order**. A transaction with a `<T commit>` in the log → **redo**; one without → **undo**.

Checkpoints

Redoing the *entire* log after every crash is impossibly slow. A **checkpoint** is a point at which the DBMS **flushes all buffers to disk and writes a `<checkpoint>` record**, so recovery need only consider transactions active **at or after** the last checkpoint. On restart, scan back to the last checkpoint and build two lists: **REDO** (committed after the checkpoint) and **UNDO** (active but not committed). Everything before the checkpoint is already safely on disk.

ARIES is the industrial algorithm — three phases: **Analysis** (find the dirty pages and active transactions from the last checkpoint), **Redo** (repeat history to reconstruct state), **Undo** (roll back losers), using an **LSN** (log sequence number) per record. Naming the three phases is enough for the exam.

Shadow paging is an alternative that keeps a **shadow (old) copy** of the page table; commit atomically switches to the new page table, and recovery just reverts to the shadow — no log, but poor concurrency and fragmentation.

Types of failure

Failure	Recovery
Transaction failure (logical error, deadlock abort)	Roll back that transaction (undo)
System crash (power/OS/DBMS)	Log-based redo/undo from last checkpoint
Media/disk failure	Restore from backup + roll forward the archived log

Likely exam questions

- [5] State the Write-Ahead Logging rule. Why is it necessary?
- [5] Differentiate between deferred and immediate database modification.
- [5] What is a checkpoint? How does it speed up recovery?
- [10] Explain log-based recovery. Describe deferred and immediate update with the undo/redo actions and the role of checkpoints.

MCQ traps

Trap	Correct answer
"The log is written before the data — this is?"	Write-Ahead Logging (WAL) .
"Deferred update needs which operation on recovery?"	REDO only (no undo).
"Immediate update needs?"	UNDO and REDO .
"A transaction with no commit record is?"	Undone .
"Checkpoints reduce?"	The amount of log to redo/undo on recovery.
"ARIES phases?"	Analysis, Redo, Undo .
"Which recovery method uses no log?"	Shadow paging .

16. Database security, authorization and SQL injection

Concept

Database security protects the database against three threats: **loss of confidentiality** (unauthorised disclosure), **loss of integrity** (improper modification) and **loss of availability** (denial of access). It rests on **authentication** (proving *who* you are), **authorization** (deciding *what* you may do), **access control** and **auditing**.

Term	Meaning
Authentication	Verifying identity — login/password, certificates, biometrics
Authorization	Granting rights (privileges) to authenticated users

Access control	Enforcing those rights — DAC, MAC, RBAC (see below)
Auditing	Logging who did what, for accountability and forensics
Encryption	Protecting data at rest and in transit (TDE, column encryption)

Access-control models:

- **DAC** (Discretionary) — the object's owner grants/revokes privileges (SQL's **GRANT/REVOKE**).
- **MAC** (Mandatory) — system-enforced security labels/clearances (Bell-LaPadula); used in high-security/government systems.
- **RBAC** (Role-Based) — privileges granted to **roles**, users assigned to roles; the practical enterprise model.

Authorization in SQL (DCL):

```
GRANT SELECT, INSERT ON STUDENT TO clerk;           -- give privileges
GRANT SELECT ON STUDENT TO analyst WITH GRANT OPTION; -- allow re-granting
REVOKE INSERT ON STUDENT FROM clerk;                -- take them back
CREATE ROLE clerk; GRANT clerk TO ravi;             -- role-based
```

Views as a security tool: grant access to a view exposing only permitted rows/columns instead of the base table — **value-independent** (columns) and **value-dependent** (rows via WHERE) authorization.

SQL injection — the named forensic topic

SQL injection is an attack that inserts malicious SQL through **unsanitised user input**, tricking the application into executing unintended commands. It is the classic web-application database vulnerability and is named explicitly in your Forensic syllabus.

How it works. An application builds a query by **string concatenation**:

```
"SELECT * FROM users WHERE name='" + input + "' AND pass='" + pw + '"
```

If the attacker enters username **' OR '1'='1' --**, the query becomes:

```
SELECT * FROM users WHERE name='' OR '1'='1' --' AND pass='...'
```

'1'='1' is always true and **--** comments out the password check, so the attacker logs in without a password. Worse payloads use **UNION SELECT** to steal other tables, or stacked queries like **'; DROP TABLE users; --**.

Types: in-band (classic/error-based, UNION-based), **blind** (boolean- or time-based, when no output is shown), and out-of-band.

Prevention — the marks are in this list:

1. **Parameterised queries / prepared statements** (bind variables) — the **single best defence**; data can never be parsed as code.
2. **Stored procedures** with parameters (if they don't themselves build dynamic SQL).
3. **Input validation and sanitisation** — whitelist allowed characters, escape special ones.
4. **Least-privilege** database accounts — the web app should not run as DBA.
5. **Web application firewall (WAF)** and error-message suppression (don't leak schema).

Parameterised example (safe):

```
PREPARE stmt FROM 'SELECT * FROM users WHERE name=? AND pass=?';  
-- the ? placeholders are bound to values, never concatenated as SQL text
```

Other DB security concerns

- **Statistical database security** — aggregate queries can leak individual data (**inference**); mitigated by query restriction and noise.
- **Data dictionary / catalog** — must itself be protected; it reveals the whole schema.
- **Auditing and logging** for non-repudiation and forensic reconstruction.

Likely exam questions

- **[5]** Differentiate between authentication and authorization.
- **[5]** Explain DAC, MAC and RBAC.
- **[5]** Explain GRANT and REVOKE with the WITH GRANT OPTION.
- **[5]** How can views be used to enforce security?
- **[5]** What is SQL injection? Give one example and two prevention techniques.
- **[10]** What is SQL injection? Explain how it works with an example, its types, and the measures to prevent it.
- **[10]** Explain database security: authentication, authorization, access-control models and the use of GRANT/REVOKE and views.

MCQ traps

Trap	Correct answer
"Proving identity is?"	Authentication . Deciding rights = authorization .
"GRANT/REVOKE implement which access model?"	DAC (discretionary).
"Best defence against SQL injection?"	Parameterised / prepared statements .
" <code>' OR '1'='1' --</code> is an example of?"	SQL injection .
"Privileges granted to roles, not users, is?"	RBAC .
"WITH GRANT OPTION lets the grantee?"	Re-grant the privilege .

17. Distributed databases, the network model and other models

Concept — Distributed databases (DDB)

A **distributed database** is a single logical database whose data is **stored across multiple physical sites** connected by a network, managed by a **DDBMS** that makes the distribution **transparent** to the user (it looks like one database).

Key techniques:

- **Fragmentation** — splitting a table: **horizontal** (rows, e.g. by region), **vertical** (columns), or **hybrid**.

- **Replication** — keeping copies of data at several sites for **availability and read performance**, at the cost of **consistency maintenance** on writes.

- **Allocation** — deciding which fragment/replica lives where.

Transparencies to name: location, fragmentation, replication, and concurrency/failure transparency.

Advantages	Disadvantages
Local autonomy and faster local access	Complexity of design and management
Reliability/availability — one site down, others serve	Costly concurrency control and recovery across sites
Scalability — add sites incrementally	Security harder over a network
Matches geographically distributed organisations	Distributed commit needed

Distributed commit uses the **two-phase commit (2PC)** protocol: a coordinator asks all sites to **prepare** (phase 1); only if **all** vote "yes" does it broadcast **commit** (phase 2), otherwise **abort** — guaranteeing atomicity across sites. (Contrast the earlier **2PL**, which is about locking, not commit — a very common confusion.)

CAP theorem (worth a line): a distributed system can guarantee at most two of **Consistency**, **Availability**, **Partition-tolerance** simultaneously.

The network model (named in the CS Degree syllabus)

The **network data model** (CODASYL/DBTG, 1971) represents data as a **graph of records connected by links called sets** (an owner–member relationship). Unlike the hierarchical model's tree, a **member record can have several owners**, so it can represent **M:N** relationships more naturally. Navigation follows **pointer chains**, so access is **procedural**.

	Hierarchical	Network	Relational
Structure	Tree	Graph	Tables
Parents per record	One	Many	(values, not links)
Relationships	1:N	1:N and M:N	via foreign keys
Navigation	Procedural (pointers)	Procedural (pointers)	Declarative (SQL)
Example	IBM IMS	CODASYL, IDMS	Oracle, MySQL

Advantages of the network model: represents M:N naturally, efficient navigation, enforces integrity via set membership. **Disadvantages:** structurally complex, no ad-hoc querying, programs are tied to the physical pointer structure (poor data independence) — which is exactly why the relational model displaced it.

Object and object-relational databases (named in the Forensic syllabus)

- **OODBMS** stores **objects** directly (attributes + methods + inheritance + object identity), good for complex data (CAD, GIS, multimedia) and eliminating the object–relational "impedance mismatch," but lacks a standard query language as mature as SQL.

- **ORDBMS** extends the relational model with **user-defined types, inheritance, and complex attributes** while keeping SQL — the pragmatic middle ground (PostgreSQL, Oracle).

Likely exam questions

- **[5]** What is a distributed database? State its advantages and disadvantages.
- **[5]** Differentiate horizontal and vertical fragmentation. What is replication?
- **[5]** Explain the network data model with an example.
- **[5]** Differentiate the hierarchical, network and relational models.
- **[10]** Explain distributed databases: fragmentation, replication, transparency and the two-phase commit protocol.

MCQ traps

Trap	Correct answer
"Splitting a table by rows is?"	Horizontal fragmentation . By columns = vertical .
"Keeping copies at several sites is?"	Replication .
"Distributed atomic commit uses?"	Two-phase commit (2PC) — not 2PL.
"The network model is also called?"	CODASYL / DBTG .
"Which model allows a record many owners?"	Network .
"CAP theorem — you can guarantee at most?"	Two of C, A, P .
"ORDBMS keeps which query language?"	SQL (extended).

18. Last-week revision sheet

Twenty facts most likely to be one-mark MCQs:

1. **Information = data + processing/context**; metadata = data about data, held in the **data dictionary/catalog**.
2. DBMS over files: removes **redundancy, inconsistency**; adds **data independence, integrity, concurrency, recovery, security**.
3. ANSI/SPARC = **3 levels**: external (view) / conceptual (logical) / internal (physical).
4. **Logical DI harder than physical DI**. Physical = add an index; logical = change the schema.
5. E-R symbols: entity = **rectangle**, weak entity = **double rectangle**, relationship = **diamond**, attribute = **ellipse**, key = **underlined**, multivalued = **double ellipse**, derived = **dashed ellipse**.
6. **Total participation = double line**; degree = number of entity types.
7. **Generalisation = bottom-up**, specialisation = top-down, **aggregation = relationship as an entity**.
8. E-R mapping: **M:N → new table**; **1:N → FK on the N side**; multivalued → new table.
9. Keys: **super ⊇ candidate ⊇ {primary, alternate}**; PK **not NULL**; FK may repeat/be NULL.

10. **Entity integrity = PK not NULL; referential integrity = FK matches a PK or is NULL.**
11. σ = **select rows**, π = **project columns**; **projection removes duplicates.**
12. **Degree = columns, cardinality = rows.**
13. Normal forms: **1NF atomic; 2NF no partial; 3NF no transitive; BCNF every determinant a super key; 4NF no MVD. BCNF stricter than 3NF; 3NF preserves dependencies.**
14. **X^+ = all attributes $\Rightarrow$ X is a super key.** Armstrong's axioms: **reflexivity, augmentation, transitivity** (sound and complete).
15. **Lossless join $\Leftrightarrow$ common attributes are a super key of one table.**
16. SQL: **DDL** (CREATE/ALTER/DROP), **DML** (SELECT/INSERT/UPDATE/DELETE), **DCL** (GRANT/REVOKE), **TCL** (COMMIT/ROLLBACK). TRUNCATE = DDL, no rollback.
17. **WHERE filters rows (no aggregates); HAVING filters groups (aggregates allowed).**
18. **ACID** = Atomicity, Consistency, Isolation, Durability; isolation via concurrency control, durability + atomicity via the log.
19. **2PL guarantees conflict serializability**; precedence graph **acyclic $\Leftrightarrow$ serializable**; **timestamp ordering is deadlock-free.**
20. **WAL: log before data.** Deferred = **redo only**; immediate = **undo + redo. Best defence against SQL injection = parameterised queries.**

Descriptive-paper strategy. DBMS reliably supplies Section A across all three papers and at least one 15-mark question you can fully answer. Prepare these six as guaranteed long answers, each with its diagram or worked steps:

- (a) **E-R diagram** for a given scenario with all attribute types, a weak entity and a generalisation — *draw it first, label it*;
- (b) **E-R $\rightarrow$ relational mapping** with primary and foreign keys;
- (c) **normalization 1NF $\rightarrow$ BCNF** with a single running example and worked decompositions — the highest-scoring, because the steps are unambiguous to mark;
- (d) **SQL joins** with example queries and output tables;
- (e) **ACID + transaction states**, and **2PL/precedence graph**;
- (f) **SQL injection** with the ' OR '1'='1' example and the prevention list.

Always draw the diagram or lay out the worked steps *first*, label them, then write the prose — if you run out of time, a labelled diagram or a correct decomposition still scores.